ЛАБОРАТОРНА РОБОТА № 2
ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набути практичні навички з використання фрагментів для інтерфейсу користувача.

Завдання

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («OK»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить текстове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

Варіант №3

Вікно містить групу опцій (радіо-батонів) для вибору шрифту, текстове поле для введення інформації та кнопки «OK» і «Cancel». Вивести введений рядок обраним шрифтом при натисканні на кнопку «OK» у деяке текстове поле та очистити при натисканні кнопки «Cancel».

Реалізація

Метою роботи було дослідити створення та взаємодію фрагментів у межах однієї діяльності та набути практичних навичок побудови інтерфейсу користувача з використанням декількох фрагментів.

У попередній лабораторній роботі інтерфейс програми складався з одного екрану (Activity), де всі елементи — група радіо-кнопок для вибору шрифту, текстове поле для введення інформації, кнопки та поле для відображення результату — знаходилися в одному XML-файлі розмітки. Уся логіка обробки подій також була реалізована в одному класі MainActivity. Результат взаємодії відображався в тому ж інтерфейсі без поділу на окремі компоненти.

У даній роботі функціональність залишилася аналогічною, однак архітектура програми була змінена відповідно до вимог завдання. Інтерфейс було розділено на два фрагменти:

- Перший фрагмент містить форму для введення даних: групу радіо-кнопок для вибору шрифту, текстове поле для введення тексту та кнопку «OK».

- Другий фрагмент містить текстове поле для відображення результату та кнопку «Cancel».

Після натискання кнопки «OK» у першому фрагменті відбувається перевірка коректності введених даних. Якщо текст не введено або шрифт не обрано, відображається спливаюче повідомлення з проханням завершити введення всіх даних. Якщо дані введені коректно, вони передаються з першого фрагмента до Activity, а звідти — до другого фрагмента, який відображає введений текст обраним шрифтом.

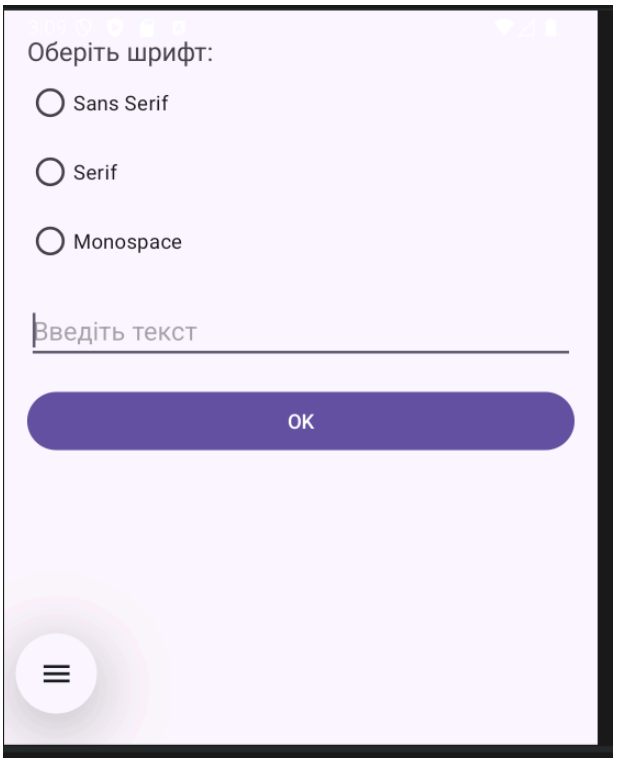

Рисунок 1 – Початковий стан програми. Відображається перший фрагмент із формою введення даних

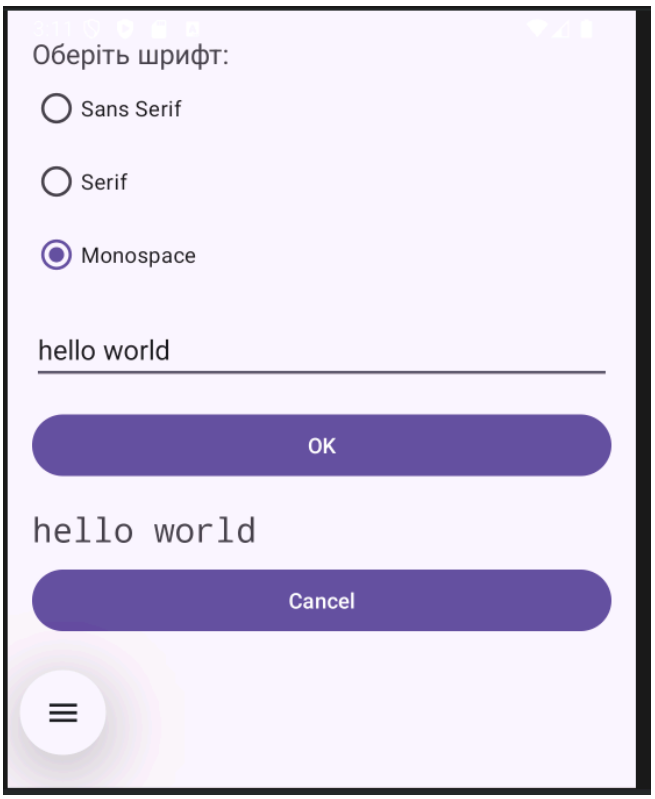

Рисунок 2 – Відображення результату у другому фрагменті після натискання кнопки «OK».

Кнопка «Cancel», розташована у другому фрагменті, виконує очищення результату та видалення (приховування) другого фрагмента з екрана. Одночасно відбувається очищення форми введення в першому фрагменті.

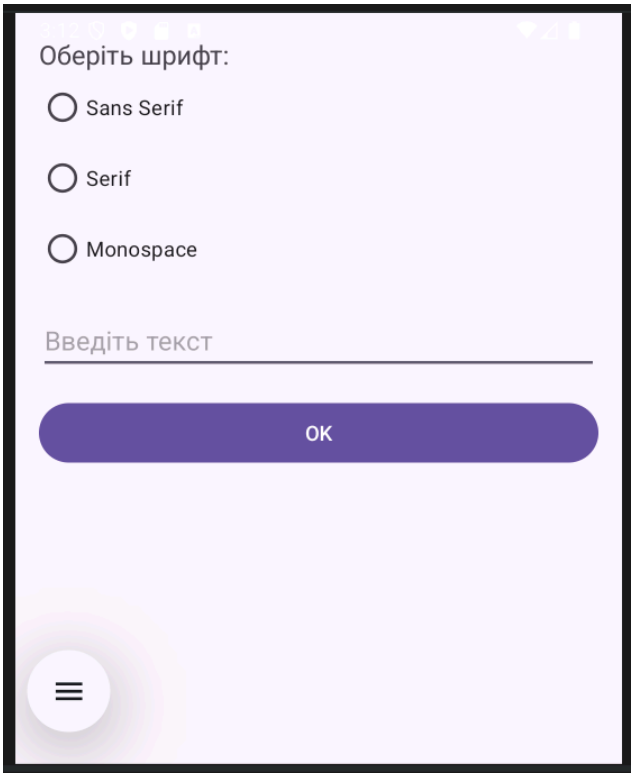

Рисунок 3 – Результат роботи кнопки «Cancel»: другий фрагмент видалено, форма введення очищена.

Таким чином, основна відмінність від попередньої роботи полягає у використанні декількох фрагментів замість одного суцільного інтерфейсу. Було реалізовано взаємодію між фрагментами в межах однієї Activity, що дозволяє краще зрозуміти принцип модульної побудови інтерфейсу та організацію передачі даних між компонентами Android-додатку.

Тестування

У процесі тестування було перевірено декілька сценаріїв роботи програми. При натисканні кнопки «OK» без введення тексту та без вибору шрифту коректно відобразилося повідомлення про необхідність заповнення всіх даних.

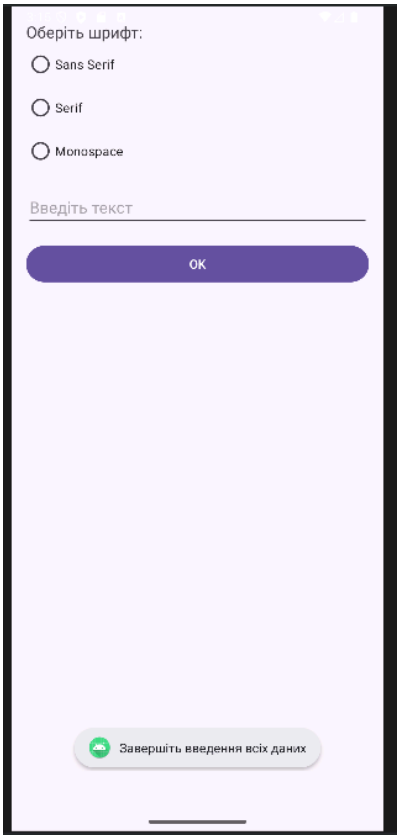

Рисунок 4 – Відображення повідомлення про необхідність введення всіх даних

Аналогічний результат спостерігався у випадку часткового введення, коли було введено лише текст або обрано лише шрифт. Після введення тексту та вибору одного з варіантів шрифту результат коректно відображається у нижньому текстовому полі з відповідним форматуванням.

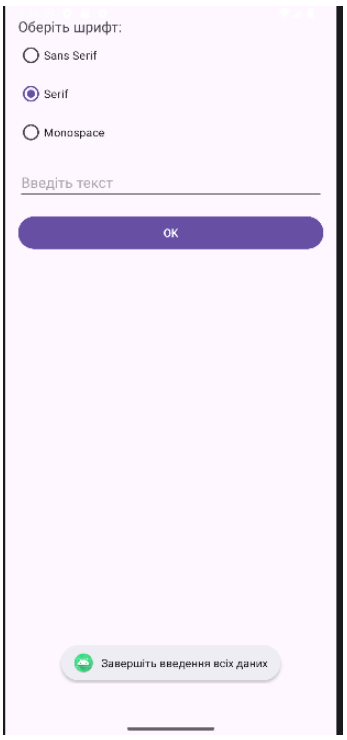

Рисунок 5 – Відображення повідомлення про необхідність введення всіх даних (текст відсутній)

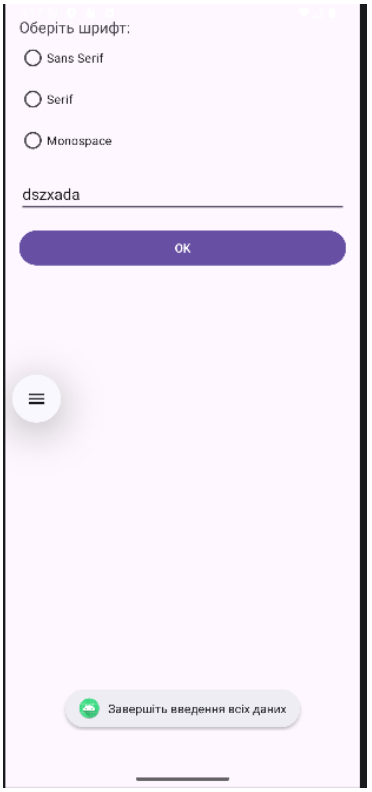

Рисунок 6 – Відображення повідомлення про необхідність введення всіх даних (шрифт відсутній)

Також коректно відображалися інші доступні шрифти при правильному введені даних.

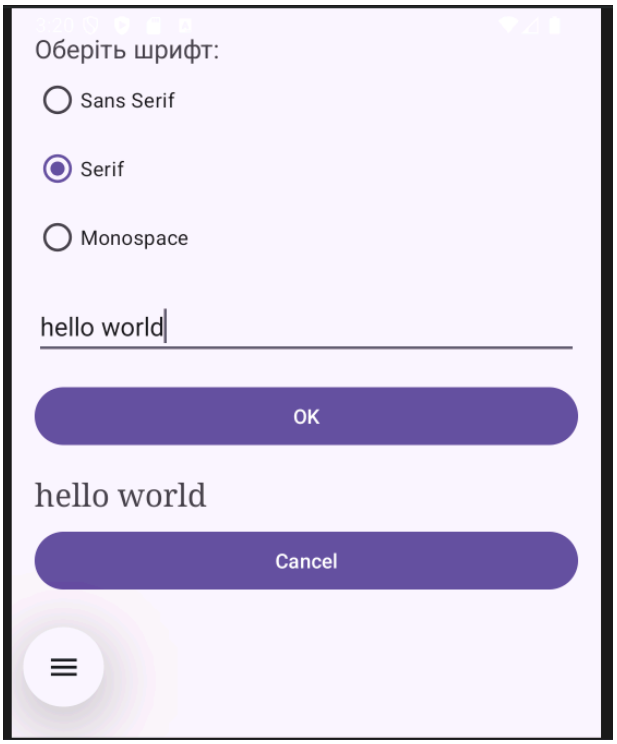

Рисунок 7 – Відображення введеного тексту зі шрифтом Serif

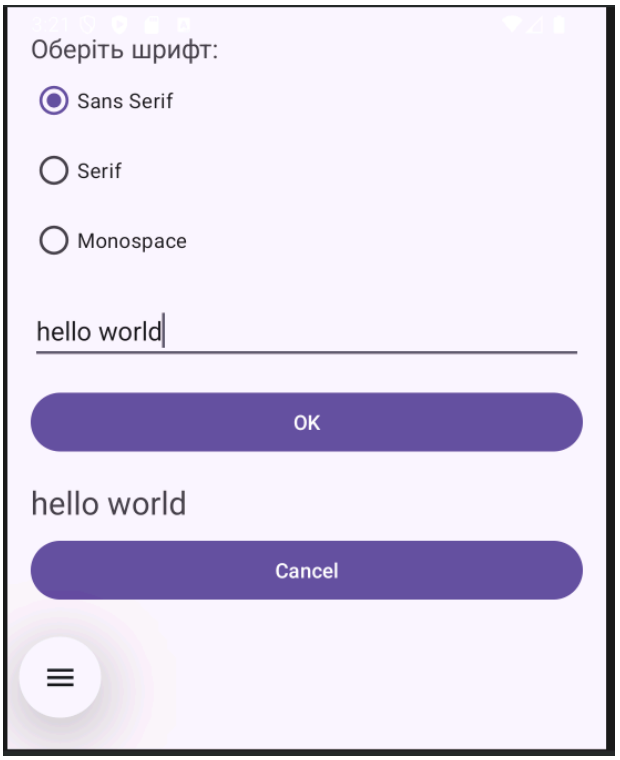

Рисунок 8 – Відображення введеного тексту зі шрифтом Sans Serif

Контрольні питання

1. Призначення та можливості компонента Fragment.

Фрагмент — це модульна частина інтерфейсу Activity, яка має власну розмітку, логіку та життєвий цикл. Він використовується для створення гнучкого та адаптивного інтерфейсу, дозволяє розділяти екран на незалежні частини, повторно використовувати UI-компоненти та спрощує підтримку коду. Фрагменти особливо важливі для планшетів і складних інтерфейсів, де потрібно динамічно змінювати частини екрана без переходу між Activity.

2. Життєвий цикл компонента Fragment.

Життєвий цикл фрагмента тісно пов'язаний із життєвим циклом Activity та включає етапи створення, відображення та знищення. Основні методи: onAttach(), onCreate(), onCreateView(), onStart(), onResume() — відповідають за створення та відображення; onPause(), onStop(), onDestroyView(), onDestroy(), onDetach() — за припинення роботи та очищення ресурсів. Fragment має окремий цикл для створення та знищення View, що відрізняє його від Activity.

3. Способи створення компонента Fragment.

Фрагмент можна створити шляхом наслідування класу Fragment із подальшим перевизначенням необхідних методів. Додавання можливе статично через XML-розмітку або динамічно програмно за допомогою FragmentManager. Найчастіше використовується динамічний спосіб, що дозволяє керувати фрагментами під час виконання програми.

4. Способи управління компонентом Fragment.

Керування фрагментами здійснюється через FragmentManager та FragmentTransaction. Можна додавати (add), замінювати (replace), видаляти (remove), приховувати (hide) або показувати (show) фрагменти, а також додавати транзакції в стек повернення (back stack). Це дозволяє динамічно змінювати інтерфейс без створення нової Activity.

5. Способи взаємодії між фрагментами.

Фрагменти можуть взаємодіяти через Activity-посередник за допомогою інтерфейсів, передавати дані через Bundle (arguments), використовувати спільну ViewModel або застосовувати Fragment Result API. Найбільш поширений спосіб — передача даних через Activity, яка координує взаємодію між фрагментами.

6. Поняття системи, малої системи та мобільної платформи.

Система — це сукупність взаємопов'язаних елементів, які працюють разом для досягнення певної мети. Мала система — це система з обмеженою кількістю компонентів, спрощеною структурою та локальним призначенням. Мобільна платформа — це програмно-апаратне середовище, що забезпечує роботу мобільних пристроїв, включає операційну систему, інструменти розробки та засоби керування ресурсами пристрою.

7. Типи мобільних застосунків.

Мобільні застосунки поділяються на нативні, кросплатформені, гібридні та веб-застосунки. Нативні створюються для конкретної платформи та забезпечують максимальну продуктивність. Кросплатформені дозволяють використовувати один код для кількох платформ. Гібридні поєднують веб-технології з нативними компонентами. Веб-застосунки працюють через браузер і не потребують встановлення.

8. Класифікація та загальна характеристика середовищ розробки мобільних застосунків.

Середовища розробки поділяються на нативні (Android Studio, Xcode), кросплатформені (Flutter, React Native, Xamarin) та веб-орієнтовані. Нативні середовища забезпечують повний доступ до можливостей платформи та високу продуктивність. Кросплатформені дозволяють скоротити час розробки та використовувати спільний код. Веб-орієнтовані підходять для створення легких мобільних рішень із використанням HTML, CSS та JavaScript.

9. Класифікація та загальна характеристика мобільних платформ.

Мобільні платформи класифікуються на відкриті (Android), закриті (iOS), пропрієтарні та за підтримкою кросплатформеної розробки. Відкриті платформи дозволяють гнучке налаштування та широку інтеграцію. Закриті платформи забезпечують підвищену безпеку та стабільність завдяки контролю екосистеми. Мобільна платформа включає операційну систему, SDK, інструменти розробки, механізми безпеки та управління ресурсами пристрою, що разом забезпечують створення, запуск і підтримку мобільних застосунків.

Висновки

У ході виконання лабораторної роботи №2 було досліджено принципи створення та використання компонента Fragment у межах однієї Activity в середовищі розробки Android. Було реалізовано мобільний застосунок, інтерфейс якого побудовано з двох фрагментів: перший фрагмент забезпечує введення даних користувачем та підтвердження дії за допомогою кнопки «OK», другий фрагмент відображає результат взаємодії та містить кнопку «Cancel» для очищення даних і видалення фрагмента результату.

У процесі роботи було засвоєно механізм динамічного додавання, заміни та видалення фрагментів за допомогою FragmentManager, а також реалізовано передачу даних між фрагментами через Activity-посередник. Окрему увагу було приділено перевірці коректності введення даних та відображенню повідомлення про помилку у випадку неповного введення.

Виконання роботи дозволило зрозуміти переваги модульної побудови інтерфейсу, зокрема можливість гнучкого керування окремими частинами екрану без створення нових Activity.

Репозиторій GitHub:
https://github.com/ProfessionalA1k0/lab2_android

Код програми:

MainActivity.kt:

```
package com.example.lab2

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity(),
   InputFragment.Listener,
   ResultFragment.Listener {

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       if (savedInstanceState == null) {
           supportFragmentManager.beginTransaction()
               .replace(R.id.containerInput, InputFragment())
               .commit()
       }
   }

   override fun onOk(text: String, fontId: Int) {
       supportFragmentManager.beginTransaction()
           .replace(R.id.containerResult,
ResultFragment.newInstance(text, fontId))
           .commit()
   }

   override fun onCancel() {


(supportFragmentManager.findFragmentById(R.id.containerInput) as?
InputFragment)
           ?.clearForm()


supportFragmentManager.findFragmentById(R.id.containerResult)?.let
{ result ->
           supportFragmentManager.beginTransaction()
               .remove(result)
               .commit()
       }
   }
}
```

InputFragment.kt:

```
package com.example.lab2

import android.content.Context
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.RadioGroup
import android.widget.Toast
import androidx.fragment.app.Fragment

class InputFragment : Fragment() {

   interface Listener {
       fun onOk(text: String, fontId: Int)
   }

   private var listener: Listener? = null

   private lateinit var rgFont: RadioGroup
   private lateinit var etInput: EditText

   override fun onAttach(context: Context) {
       super.onAttach(context)
       listener = context as? Listener
   }

   override fun onCreateView(
       inflater: LayoutInflater, container: ViewGroup?,
       savedInstanceState: Bundle?
   ): View {
       val view = inflater.inflate(R.layout.fragment_input,
container, false)

       rgFont = view.findViewById(R.id.rgFont)
       etInput = view.findViewById(R.id.etInput)
       val btnOk = view.findViewById<Button>(R.id.btnOk)

       btnOk.setOnClickListener {
           val text = etInput.text.toString()
           val selectedId = rgFont.checkedRadioButtonId

           if (text.isBlank() || selectedId == -1) {
               Toast.makeText(requireContext(), "Завершіть
введення всіх даних", Toast.LENGTH_SHORT).show()
```

```
                return@setOnClickListener
            }

            listener?.onOk(text, selectedId)
        }

        return view
    }

    fun clearForm() {
        etInput.text.clear()
        rgFont.clearCheck()
    }

    override fun onDetach() {
        listener = null
        super.onDetach()
    }
}
```

ResultFragment.kt:

```
package com.example.lab2

import android.graphics.Typeface
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.fragment.app.Fragment

class ResultFragment : Fragment() {

    interface Listener {
        fun onCancel()
    }

    private var listener: Listener? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        listener = activity as? Listener
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
```

```
    ): View {
        val view = inflater.inflate(R.layout.fragment_result,
container, false)

        val tvResult = view.findViewById<TextView>(R.id.tvResult)
        val btnCancel = view.findViewById<Button>(R.id.btnCancel)

        val text = requireArguments().getString(ARG_TEXT).orEmpty()
        val fontId = requireArguments().getInt(ARG_FONT_ID)

        val typeface = when (fontId) {
            R.id.rbSans -> Typeface.SANS_SERIF
            R.id.rbSerif -> Typeface.SERIF
            R.id.rbMono -> Typeface.MONOSPACE
            else -> Typeface.DEFAULT
        }

        tvResult.text = text
        tvResult.typeface = typeface

        btnCancel.setOnClickListener {
            listener?.onCancel()
        }

        return view
    }

    companion object {
        private const val ARG_TEXT = "text"
        private const val ARG_FONT_ID = "fontId"

        fun newInstance(text: String, fontId: Int): ResultFragment
{
            return ResultFragment().apply {
                arguments = Bundle().apply {
                    putString(ARG_TEXT, text)
                    putInt(ARG_FONT_ID, fontId)
                }
            }
        }
    }
}
```

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <FrameLayout
        android:id="@+id/containerInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <FrameLayout
        android:id="@+id/containerResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp" />
</LinearLayout>
```

fragment_input.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Оберіть шрифт:"
        android:textSize="18sp" />

    <RadioGroup
        android:id="@+id/rgFont"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <RadioButton
            android:id="@+id/rbSans"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Sans Serif" />

        <RadioButton
            android:id="@+id/rbSerif"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Serif" />

        <RadioButton
```

```
            android:id="@+id/rbMono"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Monospace" />
    </RadioGroup>

    <EditText
        android:id="@+id/etInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Введіть текст"
        android:layout_marginTop="16dp" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="OK"
        android:layout_marginTop="16dp" />
</LinearLayout>
```

fragment_result.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="22sp" />

    <Button
        android:id="@+id/btnCancel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Cancel"
        android:layout_marginTop="8dp" />
</LinearLayout>
```